

LAPORAN PERCOBAAN

JOBSHEET 5



Rafael Dimas Cahyo Laksono

254107020051

TI-1C / 23

1. Tujuan Praktikum

Setelah melakukan praktikum ini diharapkan mahasiswa mampu:

- Mahasiswa mampu membuat algoritma searching bubble sort, selection sort dan insertion sort
- Mahasiswa mampu menerapkan algoritma searching bubble sort, selection sort dan insertion sort pada program

2. Praktikum

2.1. Percobaan 1

Kode:

```
1
2 public class Sorting23 {
3
4     int[] data;
5     int jumData;
6
7     Sorting23 (int Data[], int jmlDat) {
8         jumData = jmlDat;
9         data = new int[jmlDat];
10        for (int i = 0; i < jumData; i++) {
11            data[i] = Data[i];
12        }
13    }
14
15    void bubbleSort() {
16        int temp = 0;
17        for (int i = 0; i < jumData - 1; i++) {
18            for (int j = 1; j < jumData - i; j++) {
19                if (data[j - 1] > data[j]) {
20                    temp = data[j];
21                    data[j] = data[j - 1];
22                    data[j - 1] = temp;
23                }
24            }
25        }
26    }
27
28    void SelectionSort() {
29        for (int i = 0; i < jumData - 1; i++) {
30            int min = i;
31            for (int j = i + 1; j < jumData; j++) {
32                if (data[j] < data[min]) {
33                    min = j;
34                }
35            }
36            int temp = data[i];
37            data[i] = data[min];
38            data[min] = temp;
39        }
40    }
41
42    void insertionSort() {
43        for (int i = 1; i <= data.length - 1; i++) {
44            int temp = data[i];
45            int j = i - 1;
46            while (j >= 0 && data[j] > temp) {
47                data[j + 1] = data[j];
48                j--;
49            }
50            data[j + 1] = temp;
51        }
52    }
53
54    void tampil() {
55        for (int i = 0; i < jumData; i++) {
56            System.out.print(data[i] + " ");
57        }
58        System.out.println();
59    }
60 }
61
```

```

1 public class SortingMain23 {
2     public static void main(String[] args) {
3         int a[] = {20, 10, 2, 7, 12};
4
5         int b[] = {30, 20, 2, 8, 14};
6
7         int c[] = {40, 10, 4, 9, 3};
8
9         Sorting23 dataurut1 = new Sorting23(a, a.length);
10        System.out.println("Data awal 1");
11        dataurut1.tampil();
12        dataurut1.bubbleSort();
13        System.out.println("Data sudah diurutkan dengan BUBBLE SORT (ASC)");
14        dataurut1.tampil();
15
16        Sorting23 dataurut2 = new Sorting23(b, b.length);
17        System.out.println("Data awal 2");
18        dataurut2.tampil();
19        dataurut2.SelectionSort();
20        System.out.println("Data sudah diurutkan dengan SELECTION SORT (ASC)");
21        dataurut2.tampil();
22
23        Sorting23 dataurut3 = new Sorting23(c, c.length);
24        System.out.println("Data awal 3");
25        dataurut3.tampil();
26        dataurut3.insertionSort();
27        System.out.println("Data sudah diurutkan dengan INSERTION SORT (ASC)");
28        dataurut3.tampil();
29    }
30 }

```

Hasil:

```

Data awal 1
20 10 2 7 12
Data sudah diurutkan dengan BUBBLE SORT (ASC)
2 7 10 12 20
Data awal 2
30 20 2 8 14
Data sudah diurutkan dengan SELECTION SORT (ASC)
2 8 14 20 30
Data awal 3
40 10 4 9 3
Data sudah diurutkan dengan INSERTION SORT (ASC)
3 4 9 10 40

```

Pertanyaan

1. Penggalan kode tersebut adalah untuk bubble sort, yang dimana kode tersebut untuk memeriksa apakah angka pada index sebelumnya > angka pada index saat ini. Jika kondisi tersebut sesuai, maka nilai akan ditukar dengan cara menduplikat nilai asli dari data[j] ke temp, lalu nilai dari data[j-1] akan di duplikat ke data[j]. Lalu nilai data[j] sebelumnya yang telah simpan ke temp, akan diduplikat ke data[j-1].

2.

```

        int min=i;
        for (int j = i+1; j < jumData; j++) {
            if(data[j]<data[min]){
                min=j;
            }
        }

```

3. Kondisi pada perulangan tersebut adalah untuk memastikan bahwa indeks array tidak bernilai negatif agar tidak terjadi error out of bounds, dan kondisi setelahnya apakah nilai pada posisi data[j] lebih besar daripada nilai yang sedang diurutkan yang telah diletakkan ke dalam variabel temp.
4. Tujuan dari perintah tersebut adalah untuk menggeser nilai data[j+1] ke posisi data[j] dengan cara menduplikat nilainya.

2.2. Percobaan 2

```
1 public class Mahasiswa23{
2     String nim;
3     String nama;
4     String kelas;
5     double ipk;
6
7     // Konstruktor default
8     Mahasiswa23(){
9     }
10
11    // Konstruktor berparameter
12    public Mahasiswa23(String nm, String name, String kls, double ip) {
13        nim = nm;
14        nama = name;
15        kelas = kls;
16        ipk = ip;
17    }
18
19    void tampilInformasi(){
20        System.out.println("Nama: " + nama);
21        System.out.println("NIM: " + nim);
22        System.out.println("Kelas: " + kelas);
23        System.out.println("IPK: " + ipk);
24    }
25
26 }
```

```
1 public class MahasiswaBerprestasi23{
2     Mahasiswa23 [] listMhs= new Mahasiswa23 [5];
3     int idx;
4
5     void tambah (Mahasiswa23 m){
6         if(idx<listMhs.length){
7             listMhs[idx]=m;
8             idx++;
9         } else{
10            System.out.println("data sudah penuh");
11        }
12    }
13    void tampil() {
14        for(Mahasiswa23 m:listMhs) {
15            m.tampilInformasi();
16            System.out.println("-----");
17        }
18    }
19    void bubbleSort() {
20        for (int i=0; i<listMhs.length-1; i++){
21            for (int j = 1; j<listMhs.length-i;j++){
22                if (listMhs[j].ipk>listMhs[j-1].ipk) {
23                    Mahasiswa23 tmp = listMhs[j];
24                    listMhs[j] = listMhs[j-1];
25                    listMhs[j-1] = tmp;
26                }
27            }
28        }
29    }
```

```

1 public class MahasiswaDemo23 {
2     public static void main(String[] args) {
3         MahasiswaBerprestasi23 list = new MahasiswaBerprestasi23();
4         Mahasiswa23 m1 = new Mahasiswa23("123", "Zidan", "2A", 3.2);
5         Mahasiswa23 m2 = new Mahasiswa23("124", "Ayu", "2A", 3.5);
6         Mahasiswa23 m3 = new Mahasiswa23("125", "Sofi", "2A", 3.1);
7         Mahasiswa23 m4 = new Mahasiswa23("126", "Sita", "2A", 3.9);
8         Mahasiswa23 m5 = new Mahasiswa23("127", "Miki", "2A", 3.7);
9
10        list.tambah(m1);
11        list.tambah(m2);
12        list.tambah(m3);
13        list.tambah(m4);
14        list.tambah(m5);
15
16        System.out.println("Data Mahasiswa sebelum di sorting");
17        list.tampil();
18
19        System.out.println("Data Mahasiswa setelah sorting berdasarkan IPK (DESC)");
20        list.bubbleSort();
21        list.tampil();
22
23        System.out.println("Data yang sudah terurut menggunakan SELECTION SORT (ASC)");
24        list.selectionSort();
25        list.tampil();
26    }
27 }

```

Hasil:

Data Mahasiswa sebelum di sorting

Nama: Zidan

NIM: 123

Kelas: 2A

IPK: 3.2

Nama: Ayu

NIM: 124

Kelas: 2A

IPK: 3.5

Nama: Sofi

NIM: 125

Kelas: 2A

IPK: 3.1

Nama: Sita

NIM: 126

Kelas: 2A

IPK: 3.9

Nama: Miki

NIM: 127

Kelas: 2A

IPK: 3.7

Data Mahasiswa setelah sorting berdasarkan IPK (DESC)

Nama: Sita

NIM: 126

Kelas: 2A

IPK: 3.9

Nama: Miki

NIM: 127

Kelas: 2A

IPK: 3.7

Nama: Ayu

NIM: 124

Kelas: 2A

IPK: 3.5

Nama: Zidan

NIM: 123

Kelas: 2A

IPK: 3.2

Nama: Sofi

NIM: 125

Kelas: 2A

IPK: 3.1

Pertanyaan

1. Karena pada 1 iterasi akan meletakkan nilai terbesar ke akhir array. Jadi cukup dilakukan $i-1$ karena yang diakhir sudah pasti benar.
2. Karena setiap iterasi telah meletakkan 1 nilai terbesar di posisi akhir yang tepat, maka jumlah nilai yang diperiksa akan berkurang seiring bertambahnya iterasi.
3. Perulangan akan berjalan 49 kali ($\text{listMhs.length}-1$). Artinya program akan mengurutkan nilai sebanyak 49 kali iterasi dimana setiap iterasi akan memastikan 1 nilai terbesar diletakkan di posisi akhir yang tepat.

Modifikasi:

```
1 String nim, nama, kelas;
2 double ipk;
3 for(int i=1; i<=5; i++) {
4     System.out.println("Masukkan data mahasiswa ke-" + i);
5     System.out.print("NIM: ");
6     nim = input.nextLine();
7     System.out.print("Nama: ");
8     nama = input.nextLine();
9     System.out.print("Kelas: ");
10    kelas = input.nextLine();
11    System.out.print("IPK: ");
12    ipk = Double.parseDouble(input.nextLine());
13    System.out.println("-----");
14    Mahasiswa23 m = new Mahasiswa23(nim, nama, kelas, ipk);
15    list.tambah(m);
16 }
```

Percobaan 3

```
1 void selectionSort() {
2     for (int i=0; i<listMhs.length-1; i++) {
3         int idxMin = i;
4         for (int j=i+1; j<listMhs.length; j++) {
5             if (listMhs[j].ipk<listMhs[idxMin].ipk) {
6                 idxMin = j;
7             }
8         }
9         Mahasiswa23 tmp = listMhs[idxMin];
10        listMhs[idxMin] = listMhs[i];
11        listMhs[i] = tmp;
12    }
13 }
```

```
1 System.out.println("Data Mahasiswa yang sudah terurut menggunakan SELECTION SORT (ASC)");
2 list.selectionSort();
3 list.tampil();
4 }
```

Pertanyaan

Blok kode tersebut berfungsi untuk mengidentifikasi indeks dengan nilai IPK terendah pada bagian array yang belum diproses . Melalui perulangan j, setiap elemen akan divalidasi dan variabel idxMin akan memperbarui posisinya jika ditemukan IPK yang lebih kecil . Di akhir setiap fase, elemen pada idxMin ditukar dengan elemen ke-i guna memastikan data tersusun secara ascending.

Percobaan 4

```
1 void insertionSort() {
2     for (int i = 1; i < listMhs.length; i++) {
3         Mahasiswa23 temp = listMhs[i];
4         int j = i-1;
5         while (j >= 0 && listMhs[j].ipk > temp.ipk) {
6             listMhs[j+1] = listMhs[j];
7             j--;
8         }
9         listMhs[j+1] = temp;
10    }
11 }
```

```
1 System.out.println("Data yang sudah terurut menggunakan INSERTION SORT (ASC)");
2 list.insertionSort();
3 list.tampil();
```

Modifikasi Descending

```
1 void insertionSort() {
2     for (int i = 1; i < listMhs.length; i++) {
3         Mahasiswa23 temp = listMhs[i];
4         int j = i;
5         while (j > 0 && listMhs[j-1].ipk < temp.ipk) {
6             listMhs[j] = listMhs[j-1];
7             j--;
8         }
9         listMhs[j] = temp;
10    }
11 }
```